OS Lab

Practical-4

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

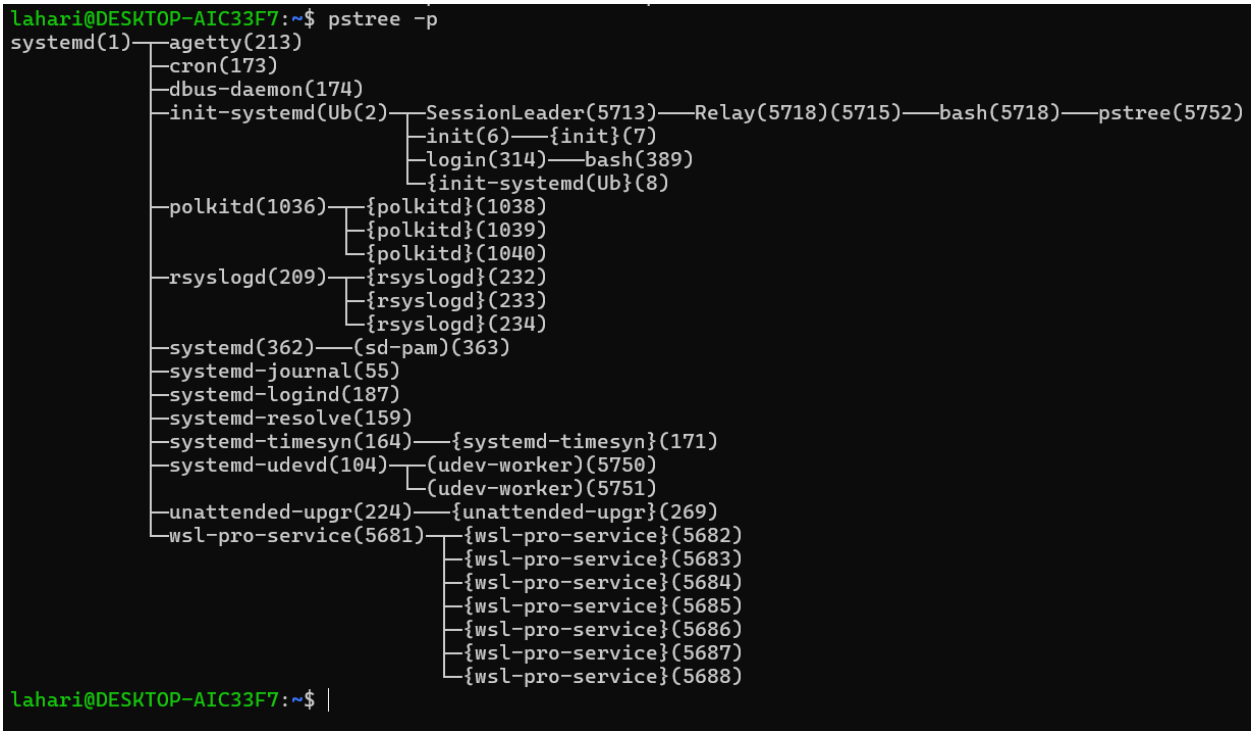
Program:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>


int main() {

    pid_t pid[3];

    int i;


    printf("Parent Process PID: %d\n", getpid());


    // Create 3 child processes

    for (i = 0; i < 3; i++) {

        pid[i] = fork();
```

```
    if (pid[i] < 0) {
        printf("Fork failed\n");
        exit(1);
    }

    if (pid[i] == 0) {
        printf("Child %d: PID = %d\n", i + 1, getpid());
        sleep(2);
        printf("Child %d completed\n", i + 1);
        exit(0);
    }
}

// wait() waits for any one child
printf("\nParent using wait()...\n");
wait(NULL);
printf("One child completed using wait()\n");

// waitpid() waits for a specific child
printf("\nParent using waitpid()...\n");
waitpid(pid[1], NULL, 0);
printf("Child 2 completed using waitpid()\n");
```

```
    // Wait for remaining child
    wait(NULL);

    printf("\nAll child processes completed.\n");

    return 0;
}
```

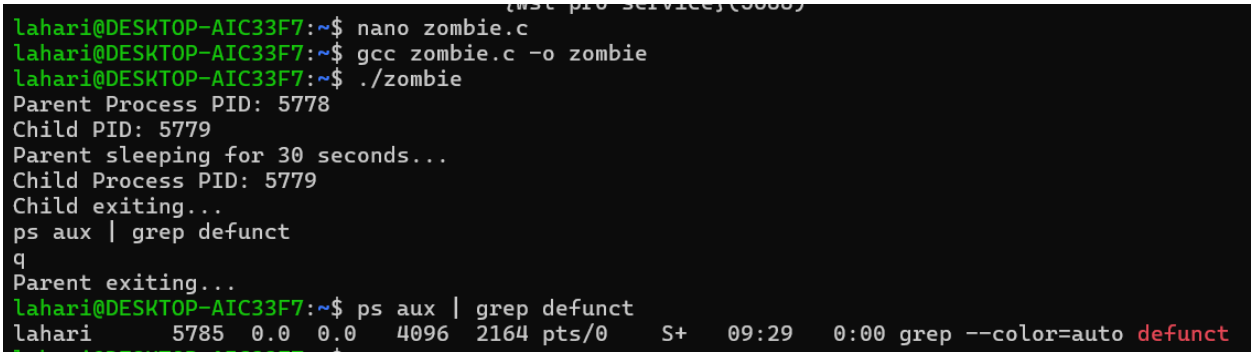
```
lahari@DESKTOP-AIC33F7:~$ nano process.c
lahari@DESKTOP-AIC33F7:~$ gcc process.c -o process
lahari@DESKTOP-AIC33F7:~$ ./process
Parent Process PID: 5743
Child 1: PID = 5744
Child 2: PID = 5745

Parent using wait()...
Child 3: PID = 5746
Child 1 completed
Child 2 completed
Child 3 completed
One child completed using wait()

Parent using waitpid()...
Child 2 completed using waitpid()

All child processes completed.
lahari@DESKTOP-AIC33F7:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
lahari      5718    5715  0 09:22 pts/0    00:00:00 -bash
lahari      5749    5718 99 09:23 pts/0    00:00:00 ps -f
```



Zombie Process

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>


int main() {

    pid_t pid;


    pid = fork();


    if (pid < 0) {

        printf("Fork failed\n");

        return 1;
```

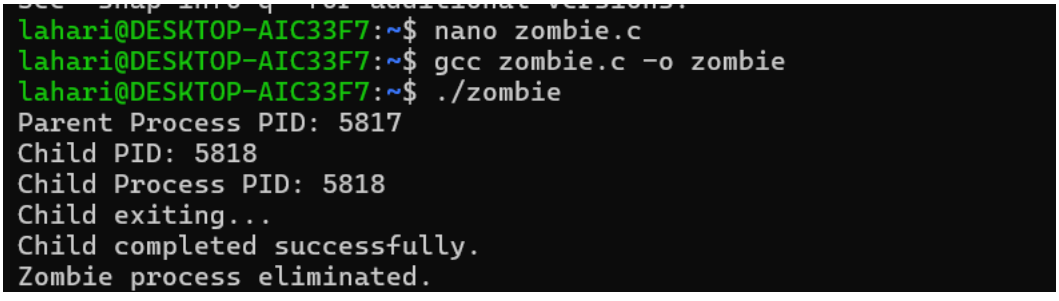
```
    }

    if (pid == 0) {
        printf("Child Process PID: %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else {
        printf("Parent Process PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        printf("Parent sleeping for 30 seconds...\n");
        sleep(30);

        printf("Parent exiting...\n");
    }

    return 0;
}
```

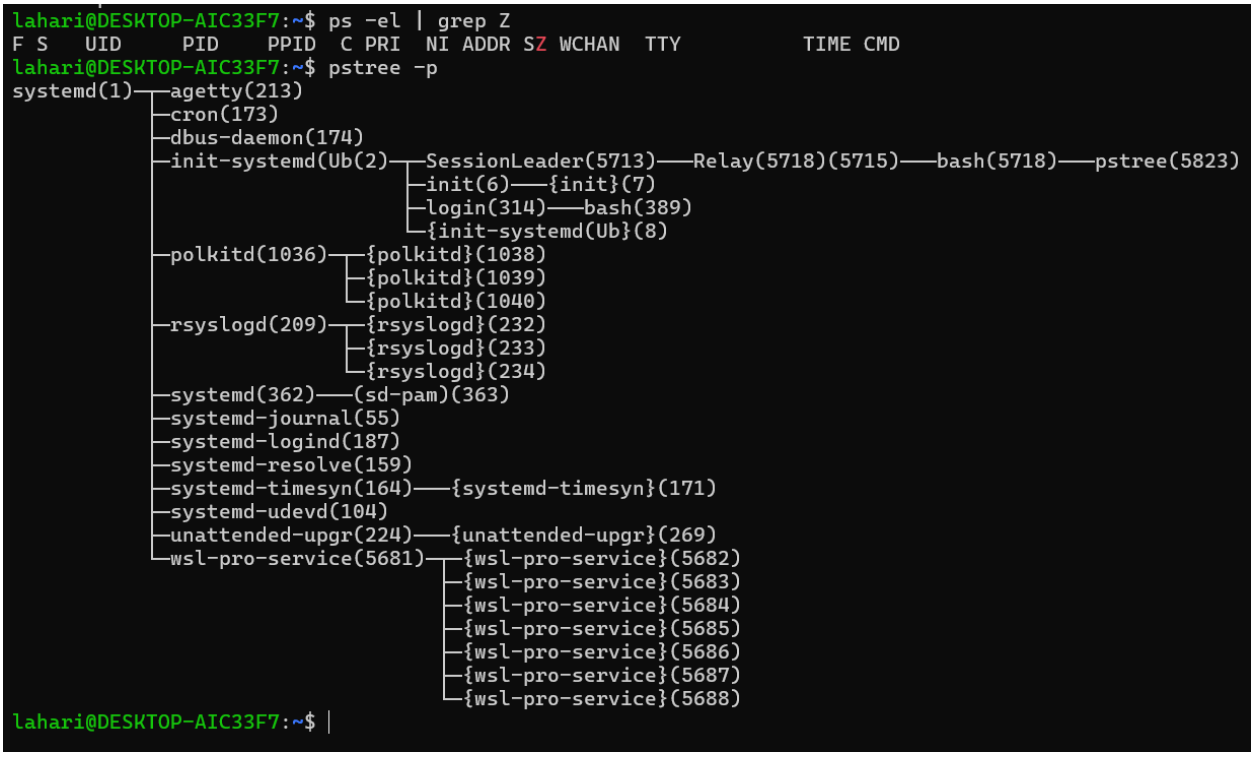


Remove the Zombie Using waitpid():

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>


int main() {

    pid_t pid;


    pid = fork();


    if (pid < 0) {

        printf("Fork failed\n");

        return 1;

    }


    if (pid == 0) {

        printf("Child Process PID: %d\n", getpid());
```

```
        printf("Child exiting...\n");
        exit(0);
    }
    else {
        printf("Parent Process PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);

        // Parent waits for the specific child
        waitpid(pid, NULL, 0);

        printf("Child completed successfully.\n");
        printf("Zombie process eliminated.\n");
    }

    return 0;
}
```




lahari@DESKTOP-AIC33F7:~$ ps -el | grep Z
F S UID PID PPID C PRI NI ADDR SZ WCHAN TTY TIME CMD
lahari@DESKTOP-AIC33F7:~$ pstree -p
systemd(1)─┬─agetty(213)
├─cron(173)
├─dbus-daemon(174)
├─init-systemd(Ub(2)─┬─SessionLeader(5713)───Relay(5718)(5715)───bash(5718)───pstree(5823)
├─init(6)───{init}(7)
├─login(314)───bash(389)
└─{init-systemd(Ub}(8)
├─polkitd(1036)─┬─{polkitd}(1038)
├─{polkitd}(1039)
└─{polkitd}(1040)
├─rsyslogd(209)─┬─{rsyslogd}(232)
├─{rsyslogd}(233)
└─{rsyslogd}(234)
├─systemd(362)───(sd-pam)(363)
├─systemd-journal(55)
├─systemd-logind(187)
├─systemd-resolve(159)
├─systemd-timesyn(164)───{systemd-timesyn}(171)
├─systemd-udevd(104)
├─unattended-upgr(224)───{unattended-upgr}(269)
└─wsl-pro-service(5681)─┬─{wsl-pro-service}(5682)
├─{wsl-pro-service}(5683)
├─{wsl-pro-service}(5684)
├─{wsl-pro-service}(5685)
├─{wsl-pro-service}(5686)
├─{wsl-pro-service}(5687)
└─{wsl-pro-service}(5688)
lahari@DESKTOP-AIC33F7:~$